BCG X data science cheat sheet

Syntax recall for a 90-minute Python test (pandas, NumPy, scikit-learn), plus the SQL practice. Snippets assume import numpy as np and import pandas as pd. pandas Series.std() is the sample standard deviation (ddof=1). NumPy's default is the population one (ddof=0). Month-end resampling is 'ME'. Unofficial.

Python core

Comprehensions

```
# filter sits at the end; a dict uses {k: v}; a set uses {v}
nums, pairs = [1, 2, 3, 4], [("a", 1), ("b", 2)]
[n * n for n in nums if n % 2 == 0]
{k: v for k, v in pairs}
{n % 2 for n in nums}
```

sorted with key

```
# a leading minus flips a number; a tuple breaks ties left to
right
# sorted() returns a new list; list.sort() changes the list
rows = [{"rev": 10, "name": "Bo"}, {"rev": 10, "name":
"Ann"}, {"rev": 4, "name": "Cy"}]
out = sorted(rows, key=lambda r: (-r["rev"], r["name"]))
```

Defaults and lambda

```
# a default is bound once, when the function is defined, not
on each call
def price(amount, discount=0.1):
    return amount * (1 - discount)
price(100)
(lambda r: (-r["rev"], r["name"]))
```

dict.get

```
# .get returns the default instead of raising KeyError
d = {}
for k in ["a", "b", "a"]:
    d[k] = d.get(k, 0) + 1
```

Counter and defaultdict

```
from collections import Counter, defaultdict
Counter(["a", "a", "b"]).most_common(1)  # [(value, count)]
groups = defaultdict(list)
for k, v in [("a", 1), ("a", 2), ("b", 3)]:
    groups[k].append(v)
dict(groups)  # a plain dict, if the caller asked for one
```

zip and enumerate

```
# zip stops at the shorter side; start=1 makes the first
index 1
names, scores = ["Ada", "Bo"], [10, 8, 7]
for i, (name, score) in enumerate(zip(names, scores),
start=1):
    i, name, score
```

f-strings and rounding

```
# .1f rounds for display; :, adds thousands separators
# the numeric answers on this test want three decimals
name, score, n = "Ada", 1.26, 12000
f"{name} scored {score:.1f}"
f"{n:,}"
f"{score:.3f}"
```

String methods

```
text = "  East-12, "
text.strip().lower().replace(",", "").split("-")
"east".startswith("e")
"file.png".endswith(".png")
"a-b".partition("-")  # ('a', '-', 'b')
```

Slices

```
# [start:end] is half-open; slicing off the end does not
raise
xs = [0, 1, 2, 3, 4]
xs[1:4], xs[-2:], xs[::-1], xs[10:20]
```

heapq

```
import heapq
# min-heap; put the priority first in the tuple
h = []
heapq.heappush(h, (2, "b"))
heapq.heappush(h, (1, "a"))
heapq.heappop(h)              # (1, "a")
heapq.nlargest(2, [3, 1, 4])
heapq.nsmallest(2, [3, 1, 4])
```

bisect

```
import bisect
xs = [1, 3, 3, 7]             # the list must already be
sorted
bisect.bisect_left(xs, 3)     # first index where 3 could sit
bisect.bisect_right(xs, 3)    # just after the existing 3s
bisect.insort(xs, 4)
```

itertools

```
import itertools
list(itertools.accumulate([1, 2, 3]))  # running sum
list(itertools.chain([1], [2, 3]))
list(itertools.product([0, 1], repeat=2))
# groupby only joins adjacent keys, so sort first
for key, grp in itertools.groupby(sorted(["b", "a", "a"])):
    key, list(grp)
```

deque

```
from collections import deque
d = deque(maxlen=3)
for x in (1, 2, 3, 4):
    d.append(x)     # full: the left end falls off
d.appendleft(0)     # full: the right end falls off
```

Exceptions

```
# catch the errors you expect; a bare except also hides bugs
def to_int(text):
    try:
        return int(text)
    except (ValueError, TypeError):
        return None
to_int("3"), to_int("x"), to_int(None)
```

pandas

Read and inspect

```
from io import StringIO
raw = "id,region,revenue\n1,East,10\n2,West,\n3,East,30\n"
df = pd.read_csv(StringIO(raw))
df.head()
df.shape, list(df.columns), df.dtypes
df.describe(include="all")
```

Selection

```
# parentheses around each mask; & and | beat comparison
operators
# between() includes both ends; query() uses quotes for
strings
df = pd.DataFrame({"region": ["East", "West", "East"], "rev":
[10, 30, 5], "sku": ["a", "b", "c"]})
df.loc[(df["region"].eq("East")) & (df["rev"] > 0), ["sku",
"rev"]]
df.iloc[0, 1]
df.query("region == 'East' and rev >= 10")
df.loc[df["sku"].isin(["a", "c"])]
df.loc[df["rev"].between(5, 10)]
```

Missing values

```
# group median first; a group that is all missing falls
through to the global median
df = pd.DataFrame({"segment": ["A", "A", "B", "B"], "income":
[10.0, None, None, None]})
med = df.groupby("segment")["income"].transform("median")
df["income"] =
df["income"].fillna(med).fillna(df["income"].median())
```

dtypes and dates

```
df = pd.DataFrame({"when": ["2024-01-02", "bad"], "rev":
["10", "20"]})
df["when"] = pd.to_datetime(df["when"], format="%Y-%m-%d",
errors="coerce")
df["rev"] = pd.to_numeric(df["rev"], errors="coerce")
df["when"].dt.year
df["when"].dt.month
df["when"].dt.day_name()
```

assign, where, map, apply

```
df = pd.DataFrame({"plan": ["Basic", "Pro", None], "rev":
[10, 20, 30], "cost": [4, 5, 6]})
df = df.assign(margin=lambda d: d["rev"] - d["cost"])
df["flag"] = np.where(df["rev"] >= 20, "high", "low")
df["band"] = np.select([df["rev"] < 15, df["rev"] < 25],
["s", "m"], default="l")
df["plan_n"] = df["plan"].map({"Basic": 0, "Pro": 1})  #
unseen -> NaN
df["margin2"] = df.apply(lambda r: r["rev"] - r["cost"],
axis=1)
```

groupby named agg

```
df = pd.DataFrame({"region": ["E", "E", "W"], "amount": [10,
20, 30], "order_id": [1, 2, 3]})
# as_index=False keeps the key as a column, so you can skip
reset_index
df.groupby("region", as_index=False).agg(
    revenue=("amount", "sum"),
    orders=("order_id", "count"),
    aov=("amount", "mean"),
)
```

transform vs agg vs filter

```
# transform aligns back to the original rows; agg shrinks to
one row per group
# filter keeps whole groups. pandas std is the sample one
(ddof=1); a 1-row group -> NaN
df = pd.DataFrame({"category": ["A", "A", "A", "B"],
"revenue": [10, 20, 30, 5]})
g = df.groupby("category")["revenue"]
df["share"] = df["revenue"] / g.transform("sum")
df["z"] = (df["revenue"] - g.transform("mean")) /
g.transform("std")
df.groupby("category", as_index=False).agg(total=("revenue",
"sum"))
df.groupby("category").filter(lambda part:
part["revenue"].sum() >= 50)
```

pivot_table, melt, crosstab

```
df = pd.DataFrame({"region": ["E", "E", "W"], "year": [2023,
2024, 2024], "rev": [10, 20, 5]})
wide = pd.pivot_table(df, index="region", columns="year",
values="rev", aggfunc="sum", fill_value=0)
wide = wide.reindex(columns=[2023, 2024, 2025], fill_value=0)
wide.reset_index().melt(id_vars="region", var_name="year",
value_name="rev")
pd.crosstab(df["region"], df["year"])
```

merge

```
# dedupe the right key first, or matching rows multiply
# a WHERE-style filter belongs in the result, not by dropping
the left rows you meant to keep
orders = pd.DataFrame({"order_id": [1, 2, 3], "customer_id":
[10, 10, 99], "amount": [5, 7, 8]})
customers = pd.DataFrame({"customer_id": [10, 11], "segment":
["Pro", "Basic"], "amount": [0, 0]})
orders.merge(
    customers, on="customer_id", how="left",
    suffixes=("_ord", "_cus"), indicator=True,
validate="m:1",
)
```

concat

```
# axis=0 stacks rows and takes the union of columns; missing
cells are null
a = pd.DataFrame({"id": [1], "revenue": [10]})
b = pd.DataFrame({"id": [2], "units": [3]})
both = pd.concat([a, b], ignore_index=True)
both[["id", "revenue", "units"]]
```

sort_values and rank

```
# kind='mergesort' is stable; na_position='last' keeps nulls
at the bottom
# dense rank shares a place and does not skip; head(n) after
the tie-break is top-N
df = pd.DataFrame({
    "category": ["A", "A", "A", "B"],
    "revenue": [10, 30, 30, 5],
    "product_id": ["b", "c", "a", "d"],
})
df["rk"] = df.groupby("category")
["revenue"].rank(ascending=False, method="dense")
ordered = df.sort_values(
    ["category", "revenue", "product_id"],
    ascending=[True, False, True], kind="mergesort",
)
ordered.groupby("category", as_index=False).head(2)
```

shift, diff, pct_change

```
# sort first; the first row in each group is NaN
df = pd.DataFrame({"store": ["S", "S", "S"], "month": [1, 2,
3], "rev": [10.0, 12.0, 15.0]})
df = df.sort_values(["store", "month"])
g = df.groupby("store")["rev"]
df["prev"] = g.shift(1)
df["delta"] = g.diff()
df["pct"] = g.pct_change()
```

rolling, expanding, ewm

```
# min_periods=1 keeps the early rows; adjust=False is the
recursive ewm
s = pd.Series([1.0, 2, 3, 4])
s.rolling(3, min_periods=1).mean()
s.expanding().mean()
s.ewm(span=3, adjust=False).mean()
```

resample

```
# 'ME' is month end; the old alias 'M' is gone
ts = pd.DataFrame({
    "date": pd.date_range("2024-01-15", periods=50,
freq="D"),
    "amount": 1.0,
}).set_index("date")
ts.resample("ME")["amount"].sum()
```

cumsum and cumcount

```
df = pd.DataFrame({"cat": ["A", "A", "B"], "rev": [10, 20,
5]})
df["csum"] = df.groupby("cat")["rev"].cumsum()
df["i"] = df.groupby("cat").cumcount()  # 0, 1, ... inside
each group
```

value_counts

```
s = pd.Series(["a", "a", "b", None])
s.value_counts(normalize=True, dropna=False)
```

duplicated

```
df = pd.DataFrame({"sku": ["a", "a", "b"], "rev": [1, 2, 3]})
df.duplicated(["sku"], keep="first")
df.drop_duplicates(["sku"], keep="last")
```

str accessor

```
s = pd.Series(["  Ab-1 ", None])
s.str.strip().str.lower().str.replace("-", "", regex=False)
s.str.extract(r"([A-Za-z]+)")
s.str.contains("Ab", na=False)
```

cut and qcut

```
# cut uses edges you choose; right=False makes the left edge
included
# qcut uses quantiles; log1p is log(1+x) so zeros survive
age = pd.Series([17, 18, 34, 35])
pd.cut(age, [0, 18, 35, 50, 120], labels=["u18", "18-34",
"35-49", "50+"], right=False)
pd.qcut(pd.Series([1, 2, 3, 4, 5, 6, 7, 8]), 4, labels=["Q1",
"Q2", "Q3", "Q4"])
np.log1p(pd.Series([0.0, 1, 10]))
```

explode

```
# one output row per list element; an empty list becomes a
null
df = pd.DataFrame({"id": [1, 2], "tags": [["a", "b"], []]})
df.explode("tags")
```

reset_index

```
# without as_index=False the key is the index, not a column
# reset_index() brings it back; drop=True throws the key away
df = pd.DataFrame({"region": ["E", "E", "W"], "rev": [10, 20,
5]})
totals = df.groupby("region")["rev"].sum()
totals.reset_index()
totals.reset_index(drop=True)
```

SettingWithCopy

```
# df[mask][col] = ... can write into a copy and change
nothing
# use .loc on the frame you own, or .copy() before editing a
subset
df = pd.DataFrame({"rev": [10, 30], "flag": [0, 0]})
df.loc[df["rev"] > 10, "flag"] = 1
sub = df.loc[df["rev"] > 10].copy()
sub.loc[:, "flag"] = 2
```

IQR cap

```
# fit the fences on the training rows only, then clip train
and test with those numbers
s = pd.Series([1.0, 2, 3, 4, 100])
q1, q3 = s.quantile(0.25), s.quantile(0.75)
iqr = q3 - q1
s.clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
```

Encodings

```
s = pd.Series(["Basic", "Pro", "Basic"])
s.map({"Basic": 0, "Plus": 1, "Pro": 2})
pd.get_dummies(s, dtype=int)
# target mean from the training rows only; an unseen level
gets the global train mean
train = pd.DataFrame({"city": ["a", "a", "b"], "y": [0, 1,
1]})
means = train.groupby("city")["y"].mean()
pd.Series(["a", "c"]).map(means).fillna(train["y"].mean())
```

NumPy

Creation

```
np.array([[1.0, 2], [3, 4]])
np.zeros((2, 3)), np.ones(4), np.arange(6), np.linspace(0, 1,
5)
np.eye(3)
```

reshape and axis

```
# the last axis is the one that changes fastest in a C
reshape
# keepdims=True so a reduced column still broadcasts against
the rows
x = np.arange(6).reshape(2, 3)
x.sum(axis=0)
x.sum(axis=1, keepdims=True)
x.T, x.ravel()
```

Broadcasting

```
# a (3,) vector lines up with the last axis
X = np.array([[1.0, 2], [3, 4], [5, 8]])
mu = X.mean(axis=0)
sd = X.std(axis=0, ddof=1)
(X - mu) / sd
```

Boolean masks

```
x = np.array([1, -2, 3, 0])
x[x > 0]
x[np.array([True, False, True, False])]
```

where

```
x = np.array([1, -2, 3, 0])
np.where(x > 0, x, 0)   # value if true, value if false
np.where(x > 0)         # indices only, when you pass one
argument
```

argsort and argmax

```
x = np.array([3, 1, 4, 2])
np.argsort(x)           # indices that would sort ascending
np.argsort(-x)          # descending, for numbers
np.argmax(x), np.argmin(x)
```

unique counts

```
vals, counts = np.unique(np.array([1, 1, 2, 2, 2]),
return_counts=True)
vals, counts
```

percentile

```
np.percentile(np.array([1.0, 2, 3, 4, 100]), [25, 50, 75])
np.quantile(np.array([1.0, 2, 3, 4]), 0.5)
```

Generator and seed

```
# seed a Generator; do not rely on the legacy global
np.random.seed
rng = np.random.default_rng(0)
rng.normal(0, 1, size=5)
rng.choice(5, size=3, replace=False)
```

Vectorised distance

```
pts = np.array([[0.0, 0], [1, 0], [0, 1]])
cent = np.array([[0.0, 0], [1, 1]])
dist = ((pts[:, None, :] - cent[None, :, :]) **
2).sum(axis=2)
dist.argmin(axis=1)
```

linalg

```
A = np.array([[1.0, 2], [3, 5]])
b = np.array([1.0, 2])
np.linalg.solve(A, b)
n = 4
x = np.arange(n, dtype=float)
y = 1 + 2 * x
design = np.column_stack([np.ones(n), x])
np.linalg.lstsq(design, y, rcond=None)[0]  # intercept, slope
```

mean and std

```
# np.std default ddof=0 (population). pandas Series.std()
default ddof=1 (sample)
x = np.array([1.0, 2, 3])
np.mean(x)
np.std(x)
np.std(x, ddof=1)
float(pd.Series(x).std())
```

scikit-learn

train_test_split

```
from sklearn.model_selection import train_test_split
df = pd.DataFrame({"x": range(20), "y": [0, 1] * 10})
train, test = train_test_split(df, test_size=0.25,
random_state=0, stratify=df["y"])
```

Pipeline and ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder,
StandardScaler
num = Pipeline([("imp", SimpleImputer(strategy="median")),
("sc", StandardScaler())])
cat = Pipeline([
    ("imp", SimpleImputer(strategy="most_frequent")),
    ("oh", OneHotEncoder(handle_unknown="ignore",
sparse_output=False)),
])
pre = ColumnTransformer([("num", num, ["age"]), ("cat", cat,
["city"])])
X = pd.DataFrame({
    "age": [20.0, np.nan, 40.0, 50.0],
    "city": ["a", "b", np.nan, "a"],
})
pre.fit_transform(X)
```

StandardScaler

```
# fit on train only; a zero-variance train column becomes 0
from sklearn.preprocessing import StandardScaler
train = pd.DataFrame({"a": [1.0, 2, 3], "b": [5.0, 5, 5]})
test = pd.DataFrame({"a": [4.0], "b": [5.0]})
scaler = StandardScaler().fit(train)
train[list(train.columns)] = scaler.transform(train)
test[list(test.columns)] = scaler.transform(test)
```

Linear, Ridge, Lasso, logistic

```
from sklearn.linear_model import Lasso, LinearRegression,
LogisticRegression, Ridge
X = np.array([[0.0, 0], [1, 0], [0, 1], [1, 1]])
y = np.array([0.0, 1, 1, 2])
y_cls = np.array([0, 0, 1, 1])
LinearRegression().fit(X, y).coef_
Ridge(alpha=1.0).fit(X, y).intercept_
Lasso(alpha=0.01, max_iter=5000).fit(X, y).coef_
# predict_proba[:, 1] is the positive score; predict()
already cut at 0.5
LogisticRegression(max_iter=500,
class_weight="balanced").fit(X, y_cls).predict_proba(X)[:, 1]
```

Forests, boosting, k-means

```
from sklearn.cluster import KMeans
from sklearn.ensemble import GradientBoostingClassifier,
RandomForestClassifier
X = np.array([[0.0, 0], [0, 1], [1, 0], [1, 1], [0.1, 0.2],
[0.8, 0.9]])
y = np.array([0, 0, 0, 1, 0, 1])
RandomForestClassifier(n_estimators=10, random_state=0,
class_weight="balanced").fit(X, y).predict(X)
GradientBoostingClassifier(n_estimators=10,
random_state=0).fit(X, y).predict(X)
KMeans(n_clusters=2, n_init=10,
random_state=0).fit(X).labels_
```

Cross-validation and grid search

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, KFold,
StratifiedKFold, cross_val_score
X = np.arange(8, dtype=float).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
est = LogisticRegression(max_iter=200)
cv = StratifiedKFold(n_splits=4, shuffle=True,
random_state=0)
cross_val_score(est, X, y, cv=cv, scoring="accuracy").mean()
list(KFold(n_splits=4, shuffle=True,
random_state=0).split(X))
# best_score_ is the CV score, not a fresh holdout
GridSearchCV(est, {"C": [0.1, 1.0]}, cv=4,
scoring="accuracy").fit(X, y).best_params_
```

Metrics

```
from sklearn.metrics import (
    accuracy_score, confusion_matrix, f1_score,
mean_absolute_error,
    mean_squared_error, precision_score, r2_score,
recall_score,
    roc_auc_score, root_mean_squared_error,
)
y = np.array([0, 0, 1, 1])
pred = np.array([0, 1, 1, 1])
score = np.array([0.1, 0.4, 0.8, 0.9])  # ranking score, not
the 0/1 cut
accuracy_score(y, pred)
precision_score(y, pred, zero_division=0)
recall_score(y, pred, zero_division=0)
f1_score(y, pred, zero_division=0)
roc_auc_score(y, score)  # pass predict_proba[:, 1], not
predict()
confusion_matrix(y, pred)
yt, yp = np.array([1.0, 2, 3]), np.array([1.1, 1.8, 3.4])
mean_absolute_error(yt, yp)
root_mean_squared_error(yt, yp)          # same as
mean_squared_error(...) ** 0.5
mean_squared_error(yt, yp)
r2_score(yt, yp)
```

Leakage rules

Fit the imputer, scaler, encoder, and any target mean on the training rows only. Transform the test with those fitted objects.

A feature from after the decision (a later refund, a window that includes the future) will not exist when you score.

A column that is a copy of the label, or a deterministic function of it, looks perfect offline and is useless live.

A random row split does not fix a feature that already contains the label. For dated rows, split by time.

A statistic computed on the whole table (a global target mean, a percentile fit on train+test) leaks the test into the train.

Stats and probability

Formulas

```
# Bayes P(A|B) = P(B|A)P(A)/P(B). PPV = TP/(TP+FP) is that,
with A = "truly positive".
# Var = E[X^2] - (E[X])^2. SE = sigma/sqrt(n). A 95% margin
is about 1.96*SE.
# Binomial C(n,k) p^k (1-p)^(n-k). Poisson P(0) = exp(-
lambda).
# Geometric P(T=k) = (1-p)^(k-1) p, mean 1/p. Gini = 2*AUC -
1.
# WAPE = sum(|error|) / sum(|actual|). MAPE divides by each
actual and breaks at 0.
# Adj R^2 = 1 - (1-R^2) * (n-1) / (n-p-1).
from math import comb, exp
bayes = 0.9 * 0.02 / 0.05
ev = 0 * 0.2 + 1 * 0.5 + 2 * 0.3
var = (0**2 * 0.2 + 1**2 * 0.5 + 2**2 * 0.3) - ev**2
se = 15 / (36 ** 0.5)
binom = comb(8, 2) * (0.25 ** 2) * (0.75 ** 6)
pois0 = exp(-2)
geom = (0.8 ** 3) * 0.2  # P(T=4) when p=0.2
```

Binomial, Poisson, normal

```
from scipy import stats
stats.binom.pmf(2, n=8, p=0.25)
stats.poisson.pmf(0, mu=2)
stats.norm.ppf(0.975)   # about 1.96, the two-sided 95% z
stats.norm.cdf(0)
```

CLT, standard error, interval

```
# SE = sigma / sqrt(n). Large n: mean ± z * SE. z =
norm.ppf(0.975) for 95%.
from scipy import stats
mean, sigma, n = 250.0, 15.0, 36
se = sigma / np.sqrt(n)
z = stats.norm.ppf(0.975)
(mean - z * se, mean + z * se)
```

t-test, chi-square, two proportions

```
# p is not the probability the null is true; it is the chance
of a result
# at least this extreme if the null is true. It is not an
effect size.
from scipy import stats
stats.ttest_ind([1, 2, 3, 4, 5], [2, 2, 3, 4, 6],
equal_var=False).pvalue
stats.chisquare([12, 8, 10, 10]).pvalue
p1, n1, p2, n2 = 0.40, 100, 0.30, 100
p = (p1 * n1 + p2 * n2) / (n1 + n2)
z = (p1 - p2) / np.sqrt(p * (1 - p) * (1 / n1 + 1 / n2))
2 * stats.norm.sf(abs(z))
```

A/B sample size

```
# Rule of thumb, two-sided 5%, about 80% power, n per arm:
# 16 * p * (1-p) / delta**2. Fix the metric before you look.
p, delta = 0.10, 0.02
n_per_arm = 16 * p * (1 - p) / delta**2
```

Correlation and causation

Correlation is a symmetric association. It does not pick a direction.
A cause needs a reason: randomisation, a discontinuity, or an assumption you can defend.
Conditioning on a mediator (a variable on the path from the cause to the effect) removes part of the effect you wanted.
Conditioning on a common effect can create a fake association. A control that is only a common cause is the one you want.

ML theory

Bias and variance

High bias: train and test are both weak. The model is too rigid for the pattern.
High variance: train is strong and test is weak. The model memorised the sample.
More data mainly cuts variance. A richer model cuts bias. Regularisation, fewer features, and a shallower tree cut variance.

L1 and L2

L2 (Ridge) shrinks coefficients toward 0 and keeps all of them. Correlated features share the weight.
L1 (Lasso) can set coefficients to exactly 0, so the feature list gets shorter. Of two correlated features it tends to keep one.
The penalty is on the size of the coefficient, so scale the features first or a large-unit column eats the whole penalty.

Which metric

Equal cost, balanced classes: accuracy or F1.
Rare positives and a review budget: precision at the top k, or recall at a fixed precision. Accuracy hides a majority classifier.
A ranking score: ROC-AUC, or PR-AUC when positives are rare. Gini = 2*AUC - 1.
Regression: MAE when outliers should not dominate, RMSE when large misses are expensive, R^2 against predicting the mean.
MAPE explodes when an actual is 0. WAPE = sum(|error|) / sum(|actual|) stays defined while the denominator is not 0.

Precision and recall

Precision = TP / (TP + FP): of the rows you flagged, how many were real.
Recall = TP / (TP + FN): of the real positives, how many you flagged.
Raising the threshold usually lifts precision and cuts recall. You pick the point from the cost, not from 0.5.
class_weight='balanced' (or a threshold that is not 0.5) when the rare class is the one that matters.

ROC and PR

ROC plots the true positive rate against the false positive rate. A sea of true negatives makes the false positive rate look small.
Precision-recall plots precision against recall and ignores the true negatives. Use it when positives are rare.
AUC summarises the ranking. It is not the operating point you will ship; the threshold chooses that.

Tree vs linear

Linear: readable coefficients, wants scaled numeric inputs, misses interactions unless you add them.
A tree cuts thresholds and interactions on its own, ignores monotone rescaling, and predicts a leaf constant outside the training range.
A forest averages unstable trees. Boosting fits residuals and overfits if the trees are deep or there are too many of them.

Overfitting signs

Train score far above the validation score.
A deep tree, tiny leaves, or a target encoding of a high-cardinality column with no smoothing.
A feature that would not exist at the time of the decision, added because the offline number went up.
The cure is a holdout you did not tune on, a simpler model, or more data — not another look at the test set.

Cross-validation

k-fold averages k cuts. StratifiedKFold keeps the class rate in each fold.
Shuffle only when the rows are exchangeable. Do not shuffle a time series; train on the past and test on the next window.
The score you tuned on is optimistic. Quote a holdout that the search never saw.

When to scale

Scale for linear and logistic regression, ridge and lasso, k-means, k-NN, PCA, and SVM.
Trees, random forests, and gradient boosting do not need scaled inputs.
Fit the scaler on the training fold only. A Pipeline does that per fold.

SQL

JOIN, GROUP BY, HAVING

```sql
-- WHERE filters rows before the groups. HAVING filters
groups.
-- COUNT(*) counts rows; SUM ignores nulls.
SELECT c.segment,
       ROUND(SUM(o.amount), 2) AS revenue,
       COUNT(*) AS orders
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'paid'
GROUP BY c.segment
HAVING COUNT(*) >= 2
ORDER BY c.segment;
```

Window functions

```sql
-- ROW_NUMBER never ties. RANK skips after a tie. DENSE_RANK
does not skip.
-- LAG / LEAD are null at the edge. Filter a window in an
outer query, not in WHERE.
-- ROWS is the frame you want for a running sum; RANGE also
pulls in peer rows.
SELECT category, sku, revenue,
       ROW_NUMBER() OVER (
         PARTITION BY category
         ORDER BY revenue DESC, sku
       ) AS rn,
       RANK() OVER (
         PARTITION BY category
         ORDER BY revenue DESC
       ) AS rnk,
       DENSE_RANK() OVER (
         PARTITION BY category
         ORDER BY revenue DESC
       ) AS dr,
       LAG(revenue) OVER (
         PARTITION BY category ORDER BY month
       ) AS prev,
       LEAD(revenue) OVER (
         PARTITION BY category ORDER BY month
       ) AS nxt,
       SUM(revenue) OVER (
         PARTITION BY category ORDER BY month
         ROWS BETWEEN UNBOUNDED PRECEDING
                  AND CURRENT ROW
       ) AS running
FROM sales
ORDER BY category, month;
```

CTEs and CASE

```sql
WITH paid AS (
  SELECT customer_id, amount,
         CASE WHEN amount >= 50 THEN 'big' ELSE 'small' END
AS band
  FROM orders
  WHERE status = 'paid'
)
SELECT band, COUNT(*) AS n, SUM(amount) AS revenue
FROM paid
GROUP BY band
ORDER BY band;
```

NULL and LEFT JOIN

```sql
-- col = NULL is never true. Use IS NULL. NULL = NULL is not
true.
-- A predicate on the right table in WHERE drops the
unmatched left rows.
-- Put that predicate in ON. COUNT(col) skips nulls; COUNT(*)
does not.
SELECT c.id,
       COALESCE(SUM(o.amount), 0) AS revenue,
       COUNT(o.id) AS paid_orders
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'paid'
GROUP BY c.id
ORDER BY c.id;
```

Dates

```sql
SELECT strftime('%Y-%m', ordered_on) AS ym,
       date(ordered_on, 'start of month') AS month_start,
       date(ordered_on, '+1 month') AS next_month
FROM orders
ORDER BY ordered_on;
```

Test-day tactics

Read the tests first

The visible example is the contract: column names, order, types, and what a tie should do.

Hidden tests follow that contract. Write the function signature the tests call, then make the example pass.

Print the frame

Before a join: print(df.head()), df.shape, df.dtypes, list(df.columns).

Print the row count before and after a merge. A sudden multiplication means a duplicated key.

Edges

Empty frame, a column that is all missing, a tie, a duplicated key, a category that appears only on the test side, and a divide by zero.

A one-row group makes a sample z-score NaN, because pandas std uses ddof=1.

Rounding

Round the number you return, not the inputs along the way.

Numeric answers are three decimals, accepted within 0.001.

np.std is population (ddof=0). Series.std() and DataFrame.std() are sample (ddof=1).

Time box

About 4 minutes for a numeric, 3 minutes for a multiple-choice item, 15 minutes for each coding task.

A running partial that you can submit beats a blank cell. If a model is not there, ship the split and the metric.

Senior cases

Forecasting

Name the decision and the grain first. The baseline is a seasonal naive.

Score WAPE and bias, not MAPE alone. Stockouts censor demand.

Validate with a rolling origin.

Intermittent demand: a rate and a size, then a stock quantile.

Churn

The decision is who to contact under a budget. Risk is not uplift.

Split by time. Value times save rate, minus cost. Compare with calling the riskiest customers.

Randomise before you trust a save rate.

Price and markdowns

Name the decision variable and the constraints before the model. The objective is usually contribution.

Sell-outs do not trace the demand curve. Pilot inside the move you are willing to keep.

Markdowns are a small discrete set plus a salvage value.

System design

A weekly list can be a batch. Use one feature query for training and for serving.

On failure, republish the last good scores. Log features, version, score, decision, and outcome.

A timeout falls back to rules. Shadow, then a slice, with a kill switch.